

Aula 0: Software R - básico

Silvio Costa

2026-03-09

1. Criação de séries temporais

```
set.seed(123)

# Sequência de datas: gerando 72 meses de dados
datas <- seq(as.Date("2020-01-01"), by="month", length.out=72)

# 72 valores gerados a partir de uma distribuição uniforme entre [0-100] (random uniform)
valores <- runif(72, min=0, max=100)

# Data frame
df <- data.frame(data=datas, valor=valores)

# Visualização do data frame
df
```

```
##      data      valor
## 1 2020-01-01 28.757752
## 2 2020-02-01 78.830514
## 3 2020-03-01 40.897692
## 4 2020-04-01 88.301740
## 5 2020-05-01 94.046728
## 6 2020-06-01  4.555650
## 7 2020-07-01 52.810549
## 8 2020-08-01 89.241904
## 9 2020-09-01 55.143501
## 10 2020-10-01 45.661474
## 11 2020-11-01 95.683335
## 12 2020-12-01 45.333416
## 13 2021-01-01 67.757064
## 14 2021-02-01 57.263340
## 15 2021-03-01 10.292468
## 16 2021-04-01 89.982497
## 17 2021-05-01 24.608773
## 18 2021-06-01  4.205953
## 19 2021-07-01 32.792072
## 20 2021-08-01 95.450365
## 21 2021-09-01 88.953932
## 22 2021-10-01 69.280341
## 23 2021-11-01 64.050681
## 24 2021-12-01 99.426978
## 25 2022-01-01 65.570580
## 26 2022-02-01 70.853047
```

```
## 27 2022-03-01 54.406602
## 28 2022-04-01 59.414202
## 29 2022-05-01 28.915974
## 30 2022-06-01 14.711365
## 31 2022-07-01 96.302423
## 32 2022-08-01 90.229905
## 33 2022-09-01 69.070528
## 34 2022-10-01 79.546742
## 35 2022-11-01 2.461368
## 36 2022-12-01 47.779597
## 37 2023-01-01 75.845954
## 38 2023-02-01 21.640794
## 39 2023-03-01 31.818101
## 40 2023-04-01 23.162579
## 41 2023-05-01 14.280002
## 42 2023-06-01 41.454634
## 43 2023-07-01 41.372433
## 44 2023-08-01 36.884545
## 45 2023-09-01 15.244475
## 46 2023-10-01 13.880606
## 47 2023-11-01 23.303410
## 48 2023-12-01 46.596245
## 49 2024-01-01 26.597264
## 50 2024-02-01 85.782772
## 51 2024-03-01 4.583117
## 52 2024-04-01 44.220007
## 53 2024-05-01 79.892485
## 54 2024-06-01 12.189926
## 55 2024-07-01 56.094798
## 56 2024-08-01 20.653139
## 57 2024-09-01 12.753165
## 58 2024-10-01 75.330786
## 59 2024-11-01 89.504536
## 60 2024-12-01 37.446278
## 61 2025-01-01 66.511519
## 62 2025-02-01 9.484066
## 63 2025-03-01 38.396964
## 64 2025-04-01 27.438364
## 65 2025-05-01 81.464004
## 66 2025-06-01 44.851634
## 67 2025-07-01 81.006435
## 68 2025-08-01 81.238951
## 69 2025-09-01 79.434232
## 70 2025-10-01 43.983169
## 71 2025-11-01 75.447516
## 72 2025-12-01 62.922113
```

```
# Estatísticas de cada coluna do data frame
summary(df)
```

```
##      data      valor
## Min.   :2020-01-01  Min.   : 2.461
## 1st Qu.:2021-06-23  1st Qu.:27.228
## Median :2022-12-16  Median :50.295
## Mean   :2022-12-16  Mean    :51.740
```

```
## 3rd Qu.:2024-06-08 3rd Qu.:78.981
## Max. :2025-12-01 Max. :99.427
```

```
# um data frame não é uma ts, precisa criar essa classe especial de dados!
```

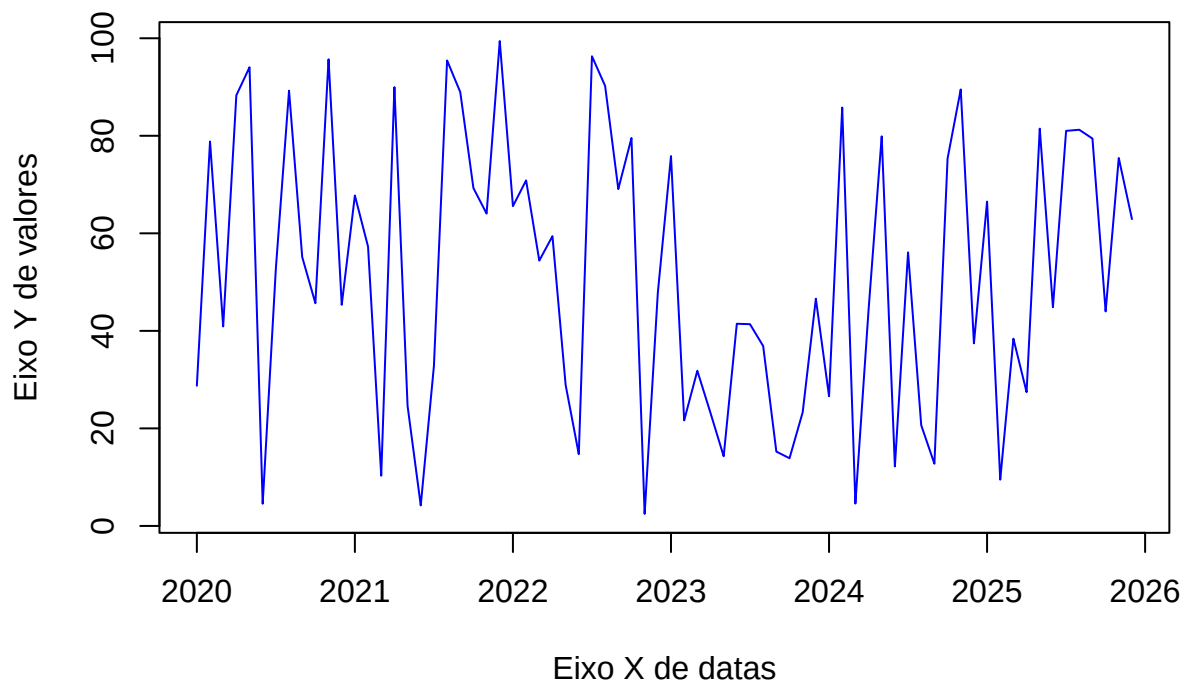
```
# Série temporal (classe ts);
```

```
serie_ts <- ts(valores, start=c(2020,1), frequency=12)
```

```
# visualizar a série temporal
```

```
plot(serie_ts, main = "Título do gráfico", xlab = "Eixo X de datas", ylab = "Eixo Y de valores", col = "blue")
```

Título do gráfico



```
summary(serie_ts)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  2.461  27.228   50.295   51.740   78.981   99.427
```

2. Alteração de frequência

```
# Agregar para anual
```

```
aggregate(serie_ts, FUN=mean)
```

```
## Time Series:
```

```
## Start = 2020
```

```
## End = 2025
```

```
## Frequency = 1
```

```
## [1] 59.93869 58.67204 56.60519 32.12365 45.42069 57.68158
```

```
# Usando o pacote zoo/xts para mudar frequência
```

```
library(zoo)
```

```
##
```

```
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric

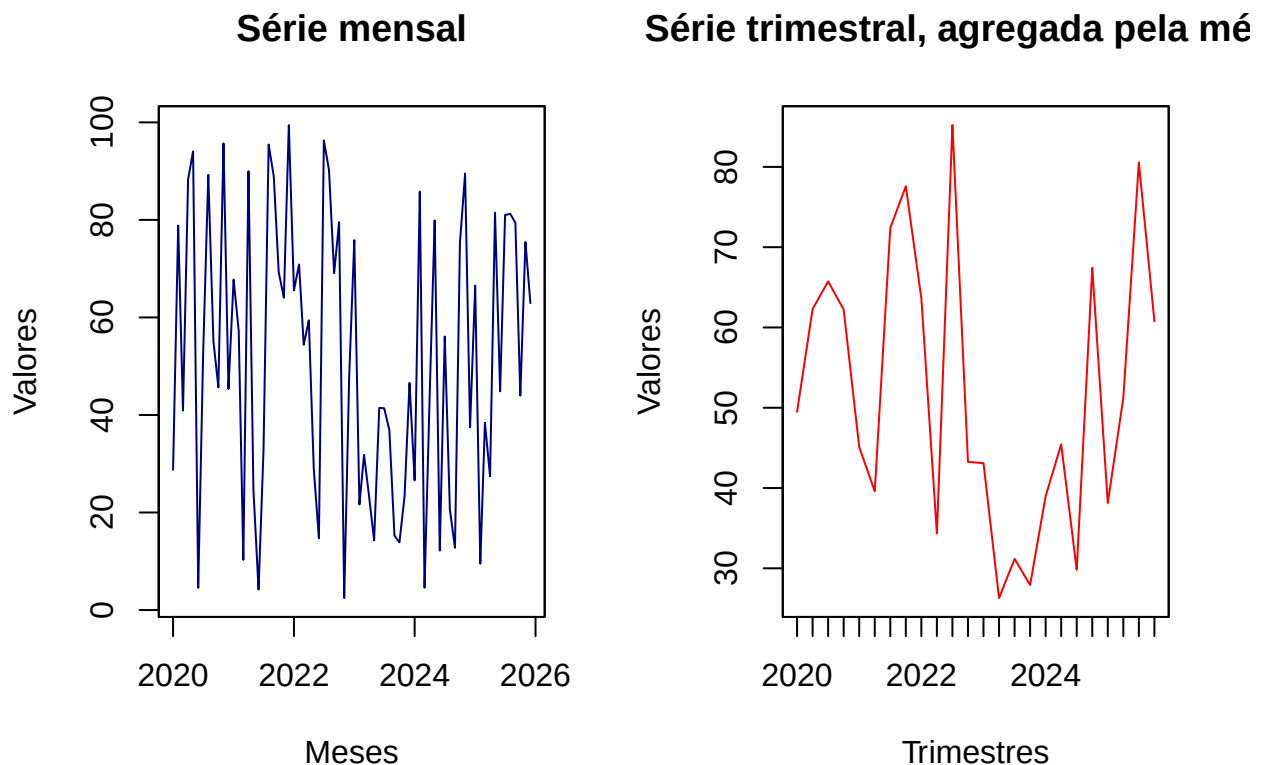
# serie_zoo mensal
serie_zoo <- zoo(valores, datas)

# serie trimestralizada pela média dos meses
serie_trimestral <- aggregate(serie_zoo, as.yearqtr, mean)

# Visualize as duas séries
par(mfrow=c(1,2))

plot(serie_zoo, xlab="Meses", ylab="Valores", main = "Série mensal", col="navy", lwd=1)

plot(serie_trimestral, xlab="Trimestres", ylab="Valores", main = "Série trimestral, agregada pela média")
```



3. Seleção e filtragem por datas

```
# formato dataframe pode ser útil para filtragem de datas e visualização (usando ggplot2, por exemplo)
# para usar as funções específicas de séries de tempo, é possível converter df em ts, xts, zoo

library(xts)
library(zoo)

df_2023 <- subset(df, format(data, "%Y") == "2023")
df_jan21 <- subset(df, format(data, "%Y-%m") >= "2021-01")
df_perodo <- subset(df, data >= as.Date("2024-06-01") & data <= Sys.Date())
```

```

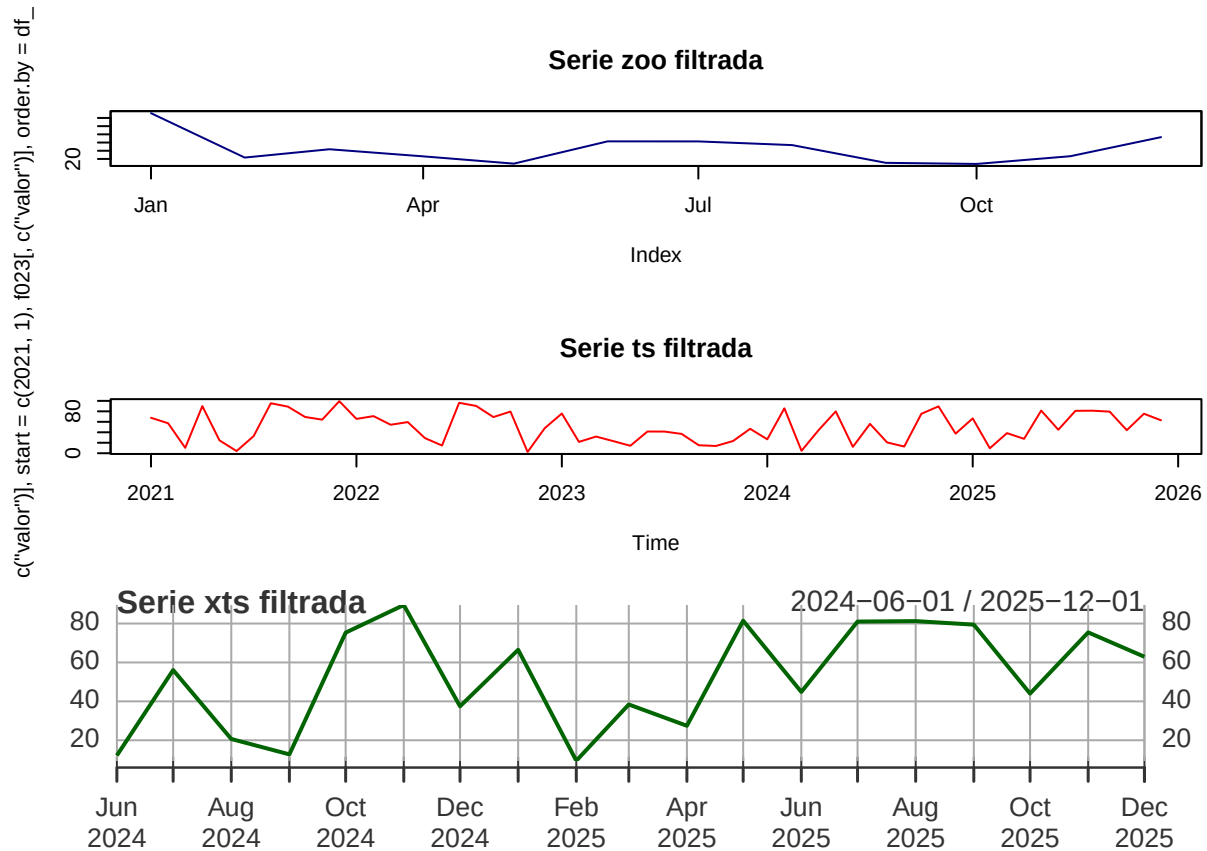
par(mfrow=c(3,1))

plot(zoo(df_2023[,c("valor")], order.by=df_2023$data), main="Serie zoo filtrada", col="navy")

plot( ts(df_jan21[, c("valor")], start = c(2021, 1), frequency = 12) , main="Serie ts filtrada", col="red")

plot(xts(df_perodo[,c("valor")], order.by=df_perodo$data), main="Serie xts filtrada", col="darkgreen")

```



4. Múltiplas séries

```

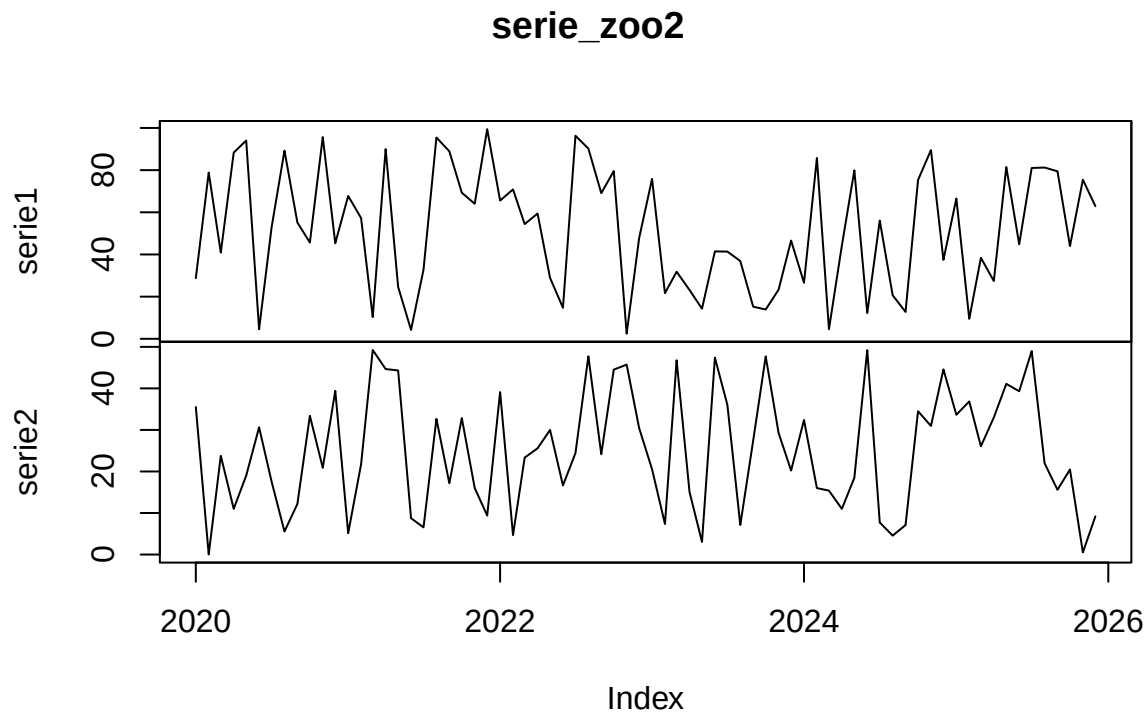
# crie uma nova sequencia de tempo
serie2 <- runif(72, 0, 50)

# crie um dataframe com serie 1 e serie 2 e mesmo vetor de datas
df_mult <- data.frame(data=datas, serie1=valores, serie2=serie2)

# converta a sequencia do dataframe em uma serie temporal zoo
serie_zoo2 <- zoo(df_mult[, -1], df_mult$data)

# visualize ambas as séries
plot(serie_zoo2)

```



5. Defasagens e diferenças

```
# use ppp::fff para acessar diretamente uma função fff específica de um pacote ppp

df$lag1 <- dplyr::lag(df$valor, 1)

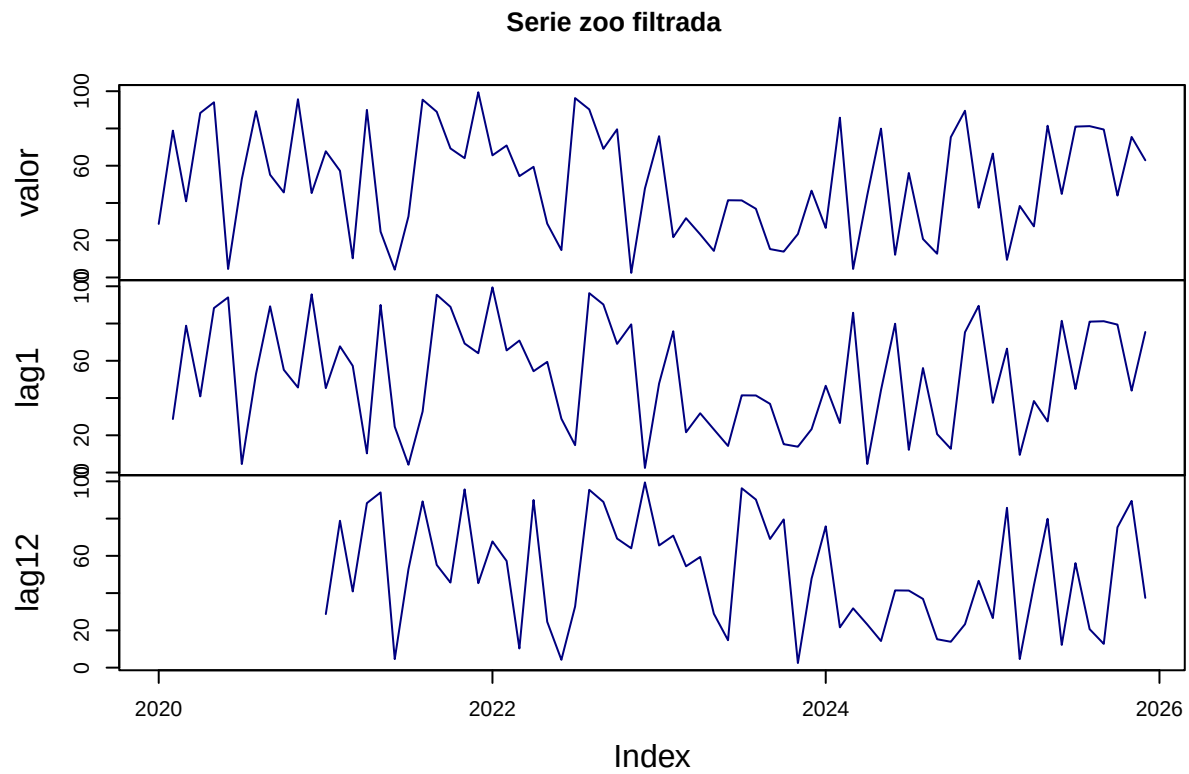
df$lag12 <- dplyr::lag(df$valor, 12)

df$diff1 <- c(NA, diff(df$valor, lag=1))

# Médias móveis
df$media_3m <- zoo::rollmean(df$valor, k=3, fill=NA)

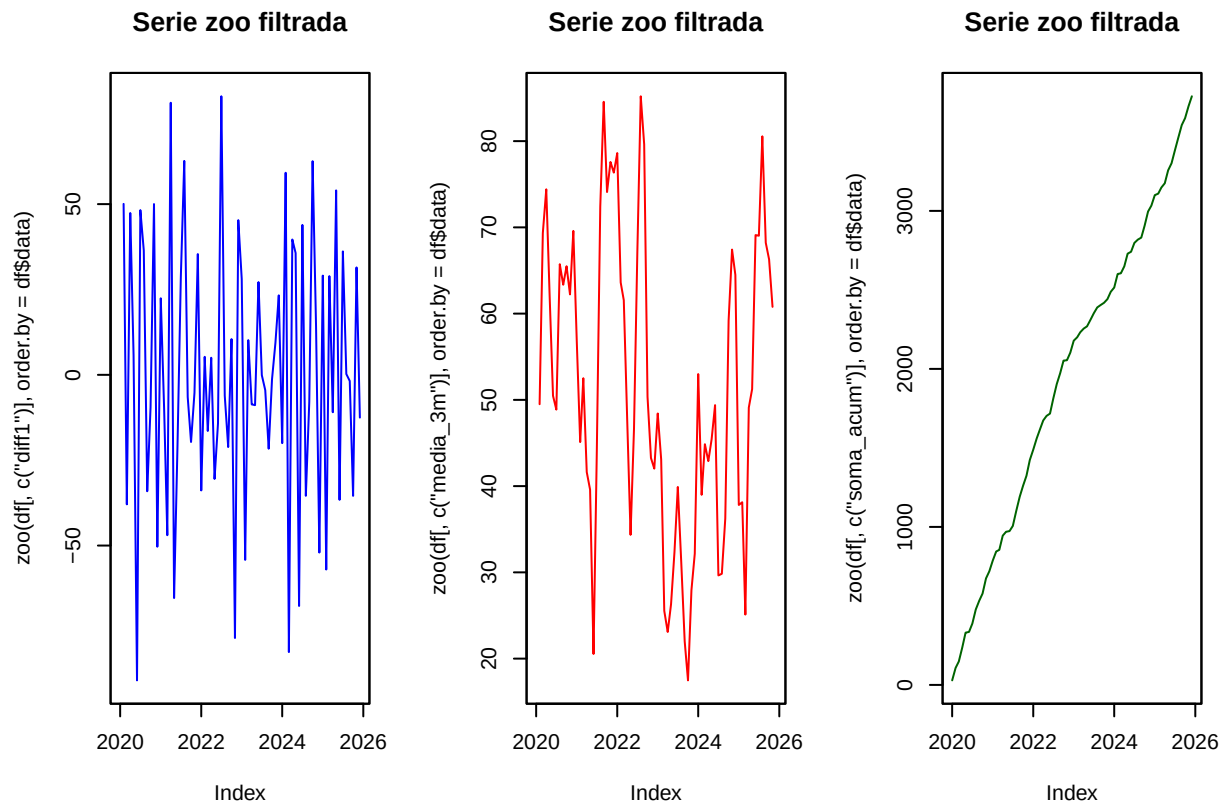
# Soma acumulada
df$soma_acum <- cumsum(df$valor)

# visualize multiplas séries de tempo convertendo diretamente do dataframe
plot(zoo(df[,c("valor", "lag1", "lag12")], order.by=df$data), main="Serie zoo filtrada", col="navy")
```



```
par(mfrow=c(1,3))

plot(zoo(df[,c("diff1")], order.by=df$data), main="Serie zoo filtrada", col="blue")
plot(zoo(df[,c("media_3m")], order.by=df$data), main="Serie zoo filtrada", col="red")
plot(zoo(df[,c("soma_acum")], order.by=df$data), main="Serie zoo filtrada", col="darkgreen")
```



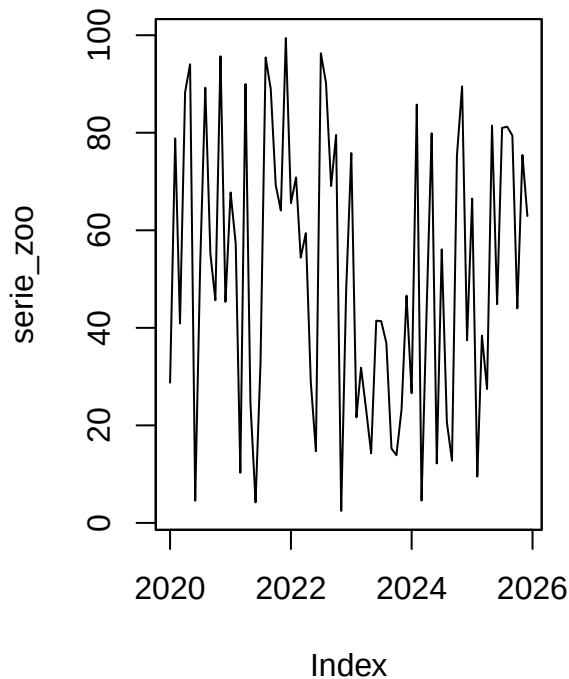
6. Reamostragem e agregação

```
library(zoo)

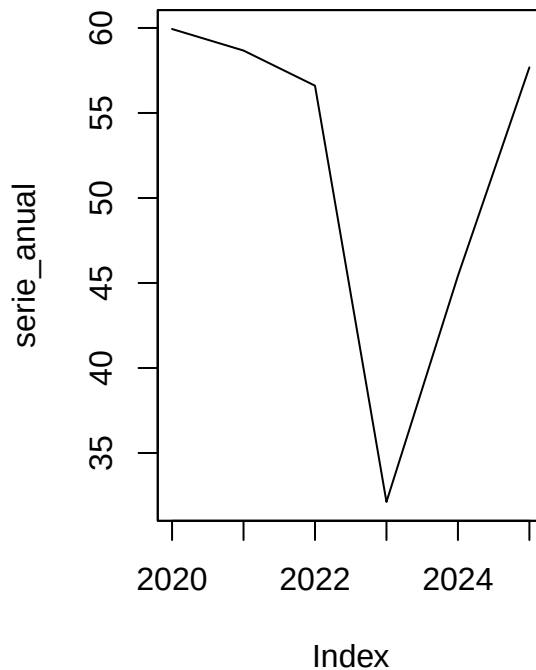
# Média anual
serie_anual <- aggregate(serie_zoo, by = format(index(serie_zoo), "%Y"), mean)

par(mfrow=c(1,2))
plot(serie_zoo, main="série original mensal")
plot(serie_anual, main="série anualizada")
```

série original mensal



série anualizada



*# o operador pipe do R cria uma sequência de operações, na qual o resultado de uma função é passada com
mutate: cria uma nova coluna chamada ano, que pega o ano de cada data na base de dados
group_by: agrupa os dados da série por ano
summarise: para cada grupo (cada ano), calcula a média da coluna valor*

```
# Agrupar por ano
library(dplyr)
```

```
##
## ##### Warning from 'xts' package #####
## #
## # The dplyr lag() function breaks how base R's lag() function is supposed to #
## # work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or #
## # source() into this session won't work correctly. #
## #
## # Use stats::lag() to make sure you're not using dplyr::lag(), or you can add #
## # conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop #
## # dplyr from breaking base R's lag() function. #
## #
## # Code in packages is not affected. It's protected by R's namespace mechanism #
## # Set `options(xts.warn_dplyr_breaks_lag = FALSE)` to suppress this warning. #
## #
## #####
##
## Attaching package: 'dplyr'
##
## The following objects are masked from 'package:xts':
##
##     first, last
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
df %>% mutate(ano=format(data, "%Y")) %>% group_by(ano) %>% summarise(media=mean(valor))

## # A tibble: 6 x 2
##   ano  media
##   <chr> <dbl>
## 1 2020   59.9
## 2 2021   58.7
## 3 2022   56.6
## 4 2023   32.1
## 5 2024   45.4
## 6 2025   57.7
```

7. Criando datas a partir de strings

```
datas_str <- c("2021-01-01", "2021-01-02")
datas <- as.Date(datas_str)
df_str <- data.frame(data=datas, valor=c(10, 20))
```

8. Criando séries simuladas

```
n <- 100
datas <- seq(as.Date("2020-01-01"), by="month", length.out=n)
tendencia <- seq(10, 50, length.out=n)
sazonal <- 10 * sin(2 * pi * (1:n) / 12)
ruído <- rnorm(n, 0, 3)

serie_sintetica <- tendencia + sazonal + ruído

serie_ts <- ts(serie_sintetica, start=c(2020,1), frequency=12)
```